

Calibrazione Sensore Sin/Cos STM32G431

<https://gemini.google.com/app/bf414486d6d477ce>

User prompt: stm32g431 devo acquisire dei valori di 2 segnali sin e cos per la calibrazione automatica di un sensore di posizione. La mia idea sarebbe di acquisire N punti a giro per M giri, senza conoscere la velocità. Il campionamento avviene a 300kHz costanti, e in ram lo spazio è limitato

Response: L'acquisizione a **300 kHz** con memoria limitata (SRAM dell'STM32G431 varia da 32 KB a 128 KB a seconda del package/versione) vieta il salvataggio continuo dei campioni raw ($300 \text{ kSPS} \times 2 \text{ canali} \times 2 \text{ byte} = 1.2 \text{ MB/s}$).

Per acquisire N punti distribuiti sull'angolo lungo M giri a velocità incognita/variabile, l'approccio corretto prevede un'estrazione in tempo reale dei dati basata su interpolazione/decimazione o l'aggiornamento online delle matrici di calibrazione.

Architettura HW/Periferiche dell'STM32G431

L'STM32G4 ha un'hardware molto potente dedicato al controllo e al processing di segnali analogici:

- **ADC1 e ADC2 in Dual Mode Sincrono:** Campionano in parallelo SIN e COS esattamente allo stesso istante (300 kHz è ampiamente dentro il limite di 4.8 MSPS).
- **DMA in Double Buffer / Circular:** Trasferisce i dati dall'ADC a piccoli buffer in SRAM (es. 256 campioni) generando un interrupt half-transfer e transfer-complete per l'elaborazione.
- **CORDIC Hardware:** L'STM32G431 integra un acceleratore CORDIC hardware in grado di calcolare $\text{atan2}(Y, X)$ e la magnitudo $\sqrt{X^2 + Y^2}$ in pochi cicli di clock, liberando la CPU.

Strategie per ovviare al problema della RAM

Tre opzioni architetturali per gestire l'acquisizione:

Opzione A: Downsampling Angolare Online (Consigliato per N punti a giro)

Invece di salvare tutti i punti nel tempo, dividi il giro ($0 \rightarrow 2\pi$) in N bin angolari (es. $N = 256$ o $N = 1024$).

1. Ad ogni campione a 300 kHz (in interrupt DMA):

- Calcoli l'angolo grezzo $\theta = \text{CORDIC_Atan2}(\text{SIN}, \text{COS})$.
- Calcoli l'indice del bin angolare: $\text{bin} = \lfloor \frac{\theta + \pi}{2\pi} \times N \rfloor$.

2. Accumulo/Filtraggio nei bin:

- In RAM mantieni solo N strutture dati con Sum_Sin, Sum_Cos e Count.
- Ad ogni campione, aggiorni: $\text{Sum_Sin}[\text{bin}] += \text{SIN_val}$, $\text{Sum_Cos}[\text{bin}] += \text{COS_val}$, $\text{Count}[\text{bin}]++$.

3. Fine dei M giri:

- La media per ogni bin $\text{Sin}_{avg}[\text{bin}] = \frac{\text{Sum_Sin}[\text{bin}]}{\text{Count}[\text{bin}]}$ fornisce N punti perfettamente equidistanti sull'angolo, indipendentemente dalla velocità di rotazione.
- **RAM occupata:** Per $N = 512 \text{ bin} \rightarrow \approx 6 \text{ KB}$ totale.

Opzione B: Calibrazione Fit ad Ellisse Online (Zero Buffer)

Se lo scopo della calibrazione è correggere **Offset** (V_{os}), **Gain Mismatch** (A_{sin}/A_{cos}) e **Phase Error** (ϕ) (modello ellittico standard per resolver/encoder magnetici):

$$X_{raw} = A_x \cos(\theta) + O_x$$

$$Y_{raw} = A_y \sin(\theta + \phi) + O_y$$

Non serve salvare i campioni. Puoi aggiornare in tempo reale a 300 kHz gli accumulatori per i minimi quadrati (Recursive Least Squares - RLS o accumulo dei momenti):

$$\sum X, \sum Y, \sum X^2, \sum Y^2, \sum X \cdot Y$$

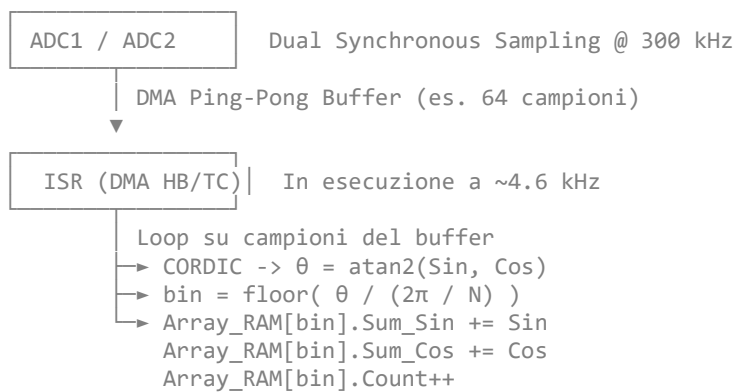
- **Procedimento:** Accumuli queste 5 variabili per M giri completi (rilevati contando i passaggi per lo zero di Atan2).
- **Fine acquisizione:** Risolvi un sistema matriciale 5×5 offline/a fine giro per estrarre direttamente i coefficienti di correzione.
- **RAM occupata:** Meno di **100 byte**.

Opzione C: Buffer Circolare con Zero-Crossing e Sottocampionamento

Se devi assolutamente registrare i campioni grezzi sequenziali per post-processing:

- Usa un trigger di start/stop basato su zero-crossing.
- Sottocampiona dinamico: calcola la velocità approssimativa nei primi 90° e imposta un divisore DMA/Timer per mantenere $N \times M$ punti totali nel buffer di RAM allocato.

Schema del Processo (Opzione A)



Note Implementative sull'STM32G431

1. **System Clock:** Configura HCLK a **170 MHz** (massima frequenza). Un interrupt DMA ogni 64-128 campioni lascia il ~90% della CPU libera.
2. **Over-sampling HW dell'ADC:** L'ADC dell'STM32G4 ha una funzione di oversampling hardware integrata. Se a 300 kHz hai rumore, puoi abilitare l'OVS hardware per guadagnare bit di ENOB senza gravare sulla CPU.
3. **Rilevamento fine M giri:** Traccia l'accumulo dell'angolo incrementale $\Delta\theta$ tra campioni consecutivi. Quando $\sum \Delta\theta \geq M \times 2\pi$, interrompi l'acquisizione.

User prompt: ah ok, quindi in caso di fit ellisse online non serve memorizzare i singoli punti, ma è sufficiente accumulare le grandezze che hai elencato?

Response: Esattamente: **non serve memorizzare i singoli punti**.

Invece di salvare migliaia di coordinate (X_i, Y_i) , aggiorni a ogni campione acquisito una serie di **momenti statistici** (gli accumulatori), riducendo l'uso di RAM da centinaia di kilobyte a **poche decine di byte**.

Però c'è un dettaglio fondamentale: l'equazione con 5 o 6 parametri

Un'ellisse generica con centro (X_0, Y_0) , semiassi a, b e inclinazione ϕ è descritta dall'equazione conica implicita:

$$AX^2 + BXY + CY^2 + DX + EY + F = 0$$

Poiché l'equazione è definita a meno di un fattore di scala globale, si impone un vincolo (es. $F = -1$ oppure $A + C = 1$) lasciando **5 parametri incogniti** da stimare.

Per trovare questi coefficienti con il metodo dei **minimi quadrati lineari** (Linear Least Squares), accumuli **6 somme principali**:

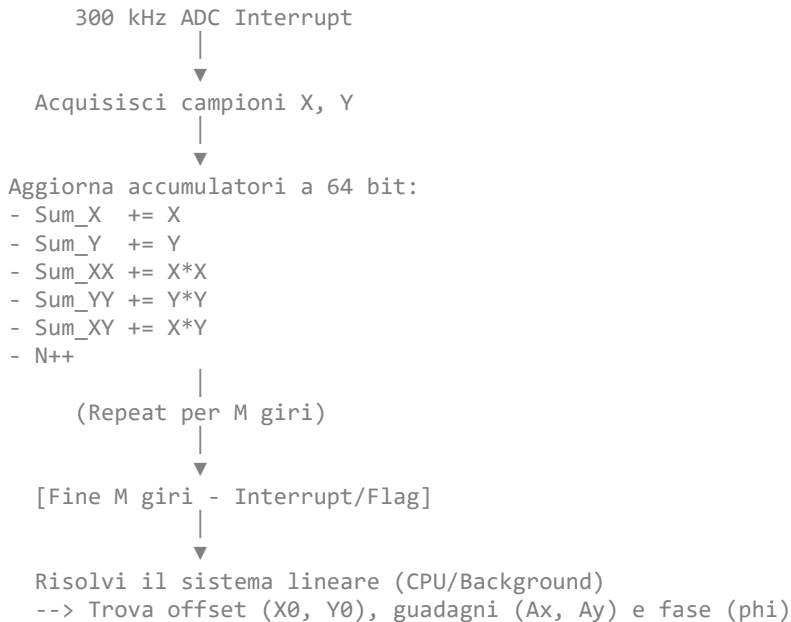
$$\sum X, \sum Y, \sum X^2, \sum Y^2, \sum XY, N \text{ (conteggio totale)}$$

Se la matrice dei normali richiede un grado di precisione o un vincolo di forma esteso (come nell'algoritmo di Direct Least Squares Fitting of Ellipses di Fitzgibbon), si accumulano anche le potenze di grado superiore:

$$\sum X^3, \sum X^2Y, \sum XY^2, \sum Y^3, \sum X^4, \dots$$

Anche nel caso più esteso, si tratta di memorizzare al massimo **10-15 variabili scalari** a 64-bit (`double` o `uint64_t` per evitare l'overflow di sommatoria).

Come funziona l'algoritmo passo-passo



- In Real-Time (Interrupt a 300 kHz / Loop DMA):** Esegui solo le moltiplicazioni e addizioni di accumulo. Con le istruzioni DSP della CPU ARM Cortex-M4F dell'STM32G4, servono pochissimi cicli di clock per ogni coppia di campioni.
- A fine misura (M giri completati):** Con le somme accumulate, componi la matrice dei sistemi normali (es. 5×5 o 6×6) e la risolvi **una sola volta** (tramite inversione di matrice o decomposizione SVD). Dalle incognite A, B, C, D, E ricavi matematicamente i parametri di calibrazione:
 - Offset X e Y:** $V_{os,x}, V_{os,y}$ (spostano il centro dell'ellisse nell'origine $(0, 0)$)
 - Guadagni A_x, A_y :** (rendono l'ellisse un cerchio)
 - Sfasamento ϕ :** (correggono la non-ortogonalità tra i due segnali)

Unico vincolo importante: Copertura del cerchio

Perché il fit ad ellisse funzioni senza salvataggio di punti:

- Il rotore deve compiere **almeno 1 giro completo** (o meglio M giri) per coprire tutti e 4 i quadranti. Se il segnale copre solo un piccolo arco (es. solo 30°), il fit ad ellisse diventa instabile e dà risultati errati.
- La velocità non conta:** Poiché sommi tutti i punti indipendentemente dal tempo, le zone in cui il motore gira più piano avranno semplicemente più punti (peso maggiore nella sommatoria), ma la forma geometrica dell'ellisse nello spazio (X, Y) rimane identica.

User prompt: e per contare gli M giri? devo calcolare l'angolo

Response: **Non serve calcolare l'angolo esatto con atan2 a ogni campione** per contare i giri. Esistono tre metodi, ordinati dal più semplice al più completo:

Metodo 1: Zero-Crossing Quadrature (Consigliato per efficienza)

Poiché i segnali Sin e Cos sono sfasati di 90° , dividono il piano cartesiano in **4 quadranti**:

Quadrante	Sin	Cos
Q1	$> V_{mid}$	$> V_{mid}$
Q2	$> V_{mid}$	$< V_{mid}$
Q3	$< V_{mid}$	$< V_{mid}$
Q4	$< V_{mid}$	$> V_{mid}$

I segnali in quadratura analogici possono essere convertiti logicamente in transizioni di quadrante per tracciare la direzione e incrementare/decrementare i giri:

Come si implementa:

- Punto medio (V_{mid}):** Si usa una stima approssimativa dell'offset (es. $V_{CC}/2 = 1.65V$, corrispondente al codice ADC 2048).
- Macchina a stati leggeri:** A ogni campione (o ad ogni blocco DMA) si determina il quadrante corrente (0, 1, 2, 3).
- Conteggio dei passi:**
 - Transizione $Q4 \rightarrow Q1$ (senso orario): `quad_count++`
 - Transizione $Q1 \rightarrow Q4$ (senso antiorario): `quad_count--`
- Conteggio giri (M):** Un giro completo equivale a **4 transizioni di quadrante** nello stesso verso ($\Delta_{quad} = \pm 4$).

Vantaggio: Bastano due confronti ($>$ o $<$) ed una logica if/else. Impatto su CPU insignificante a 300 kHz.

Metodo 2: Unwrapping dell'Angolo via CORDIC (Più preciso con inversione di marcia)

Se vuoi una misurazione continua e immune a oscillazioni o vibrazioni sulla soglia dei quadranti:

- Passi i valori Sin e Cos al **CORDIC hardware dell'STM32G431** (richiede circa 4 cicli di clock) per ottenere l'angolo grezzo $\theta_k \in [-\pi, +\pi]$.
- Calcoli il delta d'angolo tra due campioni consecutivi:

$$\Delta\theta = \theta_k - \theta_{k-1}$$

- Correggi il salto sul limite di periodo di 2π (**Phase Unwrapping**):

- Se $\Delta\theta > +\pi \implies \Delta\theta = \Delta\theta - 2\pi$
- Se $\Delta\theta < -\pi \implies \Delta\theta = \Delta\theta + 2\pi$

- Accumuli l'angolo totale:

$$\theta_{tot} = \theta_{tot} + \Delta\theta$$

- Fine misura:** Quando $|\theta_{tot}| \geq M \times 2\pi$, hai completato esattamente M giri.

Metodo 3: Conteggio basato solo sul numero di campioni N_{tot}

Se il sensore ruota a **velocità costante** impostata da banco/banco prova, è sufficiente calcolare prima il periodo o la velocità stimata contando i campioni tra due zero-crossing successivi di Sin , e fermare l'accumulo dopo N_{tot} campioni totali:

$$N_{tot} = M \times \frac{f_{sample}}{f_{rotazione}}$$

Esempio di codice per il Metodo 1 (Zero-Crossing Quadrature)

```

// Da eseguire a ogni campione nell'ISR dell'ADC/DMA
void Track_Rotations(int16_t sin_raw, int16_t cos_raw)
{
    static uint8_t prev_quad = 0;
    static int32_t quad_steps = 0;

    // Offset nominale approssimativo (es. 2048 per ADC 12-bit)
    #define V_MID 2048

    // Determina il quadrante (0, 1, 2 o 3)
    uint8_t current_quad = 0;
    if (sin_raw >= V_MID) {
        current_quad = (cos_raw >= V_MID) ? 0 : 1;
    } else {
        current_quad = (cos_raw < V_MID) ? 2 : 3;
    }

    // Calcola il cambio di quadrante
    int8_t diff = current_quad - prev_quad;

    // Gestione del passaggio per lo zero (scavalcamento tra Q3 e Q0)
    if (diff == -3) diff = 1; // Avanti: Q3 -> Q0
    else if (diff == 3) diff = -1; // Indietro: Q0 -> Q3

    quad_steps += diff;
    prev_quad = current_quad;

    // Controlla se sono stati completati M giri (M * 4 quadranti)
    if (abs(quad_steps) >= (M_TARGET_REVOLUTIONS * 4)) {
        // Stop acquisizione / Avvia calcolo ellisse
        calibration_completed = 1;
    }
}

```

User prompt: non ho capito come costruisco la matrice partendo dalle somme

Response: Per capire come si passa dalle somme alla matrice, dobbiamo guardare l'**equazione implicita dell'ellisse**:

$$AX^2 + BXY + CY^2 + DX + EY + F = 0$$

Poiché i dati di un sensore reale sono affetti da rumore, ogni punto (X_i, Y_i) inserito nell'equazione non darà esattamente zero, ma un piccolo errore e_i .

L'obiettivo dei **minimi quadrati (Linear Least Squares)** è trovare i coefficienti A, B, C, D, E, F che minimizzano la somma degli errori al quadrato:

$$S = \sum_{i=1}^N (AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i + F)^2 \longrightarrow \min$$

Step 1: Il vincolo e la riduzione dei parametri

L'equazione ha 6 coefficienti, ma sono definiti a meno di una costante moltiplicativa. Per evitare la soluzione banale (tutti i coefficienti uguali a 0), fissiamo un vincolo arbitrario, ad esempio $F = -1$:

$$AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i \approx 1$$

Definiamo il vettore dei 5 parametri incogniti da trovare:

$$\beta = \begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix}$$

Step 2: Annullamento delle derivate parziali

Per minimizzare l'errore S , si calcolano le derivate parziali rispetto a ciascuna incognita (A, B, C, D, E) e le si impone uguali a 0:

$$\frac{\partial S}{\partial A} = 0, \quad \frac{\partial S}{\partial B} = 0, \quad \frac{\partial S}{\partial C} = 0, \quad \frac{\partial S}{\partial D} = 0, \quad \frac{\partial S}{\partial E} = 0$$

Eseguendo le derivate (usando la regola della catena) e raccogliendo i termini, si ottengono **5 equazioni lineari** in 5 incognite.

Ad esempio, ponendo $\frac{\partial S}{\partial A} = 0$:

$$\sum_{i=1}^N X_i^2 (AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i - 1) = 0$$

Distribuendo la sommatoria:

$$A \sum X_i^4 + B \sum X_i^3Y_i + C \sum X_i^2Y_i^2 + D \sum X_i^3 + E \sum X_i^2Y_i = \sum X_i^2$$

Step 3: La Matrice dei Sistemi Normali $\mathbf{M} \cdot \boldsymbol{\beta} = \mathbf{V}$

Ripetendo lo stesso passaggio per tutte e 5 le derivate, si ottiene la struttura matriciale completa:

$$\begin{bmatrix} \sum X^4 & \sum X^3Y & \sum X^2Y^2 & \sum X^3 & \sum X^2Y \\ \sum X^3Y & \sum X^2Y^2 & \sum XY^3 & \sum X^2Y & \sum XY^2 \\ \sum X^2Y^2 & \sum XY^3 & \sum Y^4 & \sum XY^2 & \sum Y^3 \\ \sum X^3 & \sum X^2Y & \sum XY^2 & \sum X^2 & \sum XY \\ \sum X^2Y & \sum XY^2 & \sum Y^3 & \sum XY & \sum Y^2 \end{bmatrix} \begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix} = \begin{bmatrix} \sum X^2 \\ \sum XY \\ \sum Y^2 \\ \sum X \\ \sum Y \end{bmatrix}$$

Sintesi: Quali variabili accumulare a 300 kHz

Durante l'acquisizione dei campioni (mentre il motore gira), devi semplicemente aggiornare le **14 somme accumulate**:

Grado 1-2	Grado 3	Grado 4
$\sum X$	$\sum X^3$	$\sum X^4$
$\sum Y$	$\sum X^2Y$	$\sum X^3Y$
$\sum X^2$	$\sum XY^2$	$\sum X^2Y^2$
$\sum Y^2$	$\sum Y^3$	$\sum XY^3$
$\sum XY$		$\sum Y^4$

Step 4: Risoluzione a fine acquisizione

A fine dei M giri, l'STM32G431 deve eseguire **una sola volta**:

1. Inserisce le 14 somme nella matrice 5×5 ($\mathbf{M}$) e nel vettore noto 5×1 ($\mathbf{V}$).
2. Risolve il sistema lineare tramite eliminazione di Gauss o inversione di matrice:

$$\boldsymbol{\beta} = \mathbf{M}^{-1} \cdot \mathbf{V}$$

3. Con i coefficienti A, B, C, D, E trovati, calcola i **parametri fisici di calibrazione**:

- **Offset X:** $X_0 = \frac{2CD - BE}{B^2 - 4AC}$
- **Offset Y:** $Y_0 = \frac{2AE - BD}{B^2 - 4AC}$
- **Sfasamento (Fase):** $\phi = \frac{1}{2} \text{atan2}(B, A - C)$
- **Guadagni (Ammiezza):** Ricavati dai semiassi dell'ellisse ruotata e traslata.

User prompt: spiegami perchè azzeriamo tutte le derivate parziali

Response: Azzerare le derivate parziali è il principio fondamentale del **Metodo dei Minimi Quadrati** (*Least Squares*).

Il motivo intuitivo e matematico è che vogliamo trovare il **punto di minimo assoluto** di una funzione d'errore (chiamata anche *funzione di costo* o *residuo*).

1. L'analogia geometrica (1D)

Immagina una parabola descritta da una funzione $y = f(x)$. Il punto più basso della parabola (il suo **minimo**) è l'unico punto in cui la tangente è perfettamente orizzontale, cioè dove la pendenza (la derivata) è pari a zero:

$$\frac{df}{dx} = 0$$



2. Il problema in 5D (I nostri 5 coefficienti)

Nel caso dell'ellisse, la funzione di errore $S(A, B, C, D, E)$ non dipende da una sola variabile, ma da **5 variabili contemporaneamente** (A, B, C, D, E).

Geometricamente, S non è una parabola 2D, ma una "superficie" paraboloidica immersa in uno spazio a 6 dimensioni.

Per trovare il punto più basso di questo paraboloide (cioè la combinazione dei 5 parametri che rende l'errore totale il più piccolo possibile), dobbiamo assicurarci che la pendenza sia zero **in tutte le direzioni**.

La pendenza della funzione lungo l'asse di ogni singolo parametro è rappresentata dalla sua **derivata parziale**:

- $\frac{\partial S}{\partial A}$ è la pendenza se muoviamo solo il coefficiente A .
- $\frac{\partial S}{\partial B}$ è la pendenza se muoviamo solo il coefficiente B .
- ...e così via.

L'unico modo in cui ci troviamo esattamente nel "fondo della valle" (il minimo errore assoluto) è se la pendenza è nulla contemporaneamente in tutte e 5 le direzioni:

$$\frac{\partial S}{\partial A} = 0, \quad \frac{\partial S}{\partial B} = 0, \quad \frac{\partial S}{\partial C} = 0, \quad \frac{\partial S}{\partial D} = 0, \quad \frac{\partial S}{\partial E} = 0$$

3. Perché questo trasforma il problema in un sistema di equazioni?

La funzione di errore totale S è la somma degli errori al quadrato per tutti gli N punti acquisiti:

$$S = \sum_{i=1}^N \left(\underbrace{AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i - 1}_{\text{Errore } e_i \text{ del singolo punto}} \right)^2$$

Quando applichiamo l'operatore di derivata parziale rispetto a uno dei parametri (ad esempio A), sfruttiamo la regola della catena $\frac{d}{dx}[u^2] = 2u \cdot u'$:

$$\frac{\partial S}{\partial A} = \sum_{i=1}^N 2 \cdot (AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i - 1) \cdot \frac{\partial}{\partial A} (AX_i^2 + \dots)$$

Poiché la derivata di AX_i^2 rispetto ad A è semplicemente X_i^2 :

$$\frac{\partial S}{\partial A} = 2 \sum_{i=1}^N (AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i - 1) \cdot X_i^2$$

Uguagliando a **zero** (e dividendo per 2):

$$\sum_{i=1}^N (AX_i^2 + BX_iY_i + CY_i^2 + DX_i + EY_i - 1) \cdot X_i^2 = 0$$

Distribuendo la sommatoria su ogni termine, le variabili X_i e Y_i (che sono i punti misurati dal tuo ADC) diventano le **somme note** ($\sum X^4, \sum X^3Y, \dots$), mentre A, B, C, D, E rimangono le uniche **incognite**.

Azzerando tutte e 5 le derivate parziali, otteniamo **5 equazioni lineari in 5 incognite**, che è l'esatto numero di equazioni necessarie per poter risolvere la matrice univocamente.

User prompt: Fornisci una funzione C efficiente per la risoluzione del sistema lineare 5x5 con Eliminazione di Gauss per STM32.

Response: Ecco un'implementazione efficiente in C dell'Eliminazione di Gauss con Pivot Parziale per risolvere un sistema lineare 5×5 ($\mathbf{M} \cdot \mathbf{x} = \mathbf{V}$).

L'algoritmo include il pivot parziale per evitare divisioni per zero e minimizzare gli errori di arrotondamento numerico nei calcoli in virgola mobile (float o double).

Codice C

```
#include <stdint.h>
#include <math.h>

#define MATRIX_SIZE 5

/**
 * @brief Risolve il sistema lineare M * x = V di dimensione 5x5 usando
 *        l'Eliminazione di Gauss con Pivot Parziale.
 *
 * @param M Matrice 5x5 dei coefficienti (somme accumulate). Verrà modificata.
 * @param V Vettore dei termini noti di dimensione 5. Verrà modificato.
 * @param x Vettore di output di dimensione 5 contenente le incognite [A, B, C, D, E].
 * @return int8_t 0 se la risoluzione ha successo, -1 se la matrice è singolare.
 */
int8_t Solve_System_5x5(float M[MATRIX_SIZE][MATRIX_SIZE], float V[MATRIX_SIZE], float x[MATRIX_SIZE])
{
    float A[MATRIX_SIZE][MATRIX_SIZE + 1];

    // 1. Costruzione della matrice aumentata [M | V]
    for (uint8_t i = 0; i < MATRIX_SIZE; i++) {
        for (uint8_t j = 0; j < MATRIX_SIZE; j++) {
            A[i][j] = M[i][j];
        }
        A[i][MATRIX_SIZE] = V[i];
    }

    // 2. Eliminazione Gaussiana Forward con Pivot Parziale
    for (uint8_t k = 0; k < MATRIX_SIZE; k++) {

        // Trova la riga con il valore assoluto massimo nella colonna k (Pivot)
        uint8_t max_row = k;
        float max_val = fabsf(A[k][k]);
        for (uint8_t i = k + 1; i < MATRIX_SIZE; i++) {
            if (fabsf(A[i][k]) > max_val) {
                max_val = fabsf(A[i][k]);
                max_row = i;
            }
        }

        // Controllo matrice singolare (pivot quasi nullo)
        if (max_val < 1e-9f) {
            return -1; // Sistema non risolvibile
        }

        // Scambio della riga corrente con la riga del pivot massimo
        if (max_row != k) {
```

```

        for (uint8_t j = k; j <= MATRIX_SIZE; j++) {
            float temp = A[k][j];
            A[k][j] = A[max_row][j];
            A[max_row][j] = temp;
        }
    }

    // Annullamento dei termini sotto il pivot
    for (uint8_t i = k + 1; i < MATRIX_SIZE; i++) {
        float factor = A[i][k] / A[k][k];
        for (uint8_t j = k; j <= MATRIX_SIZE; j++) {
            A[i][j] -= factor * A[k][j];
        }
    }
}

// 3. Sostituzione all'indietro (Backward Substitution)
for (int8_t i = MATRIX_SIZE - 1; i >= 0; i--) {
    float sum = A[i][MATRIX_SIZE];
    for (uint8_t j = i + 1; j < MATRIX_SIZE; j++) {
        sum -= A[i][j] * x[j];
    }
    x[i] = sum / A[i][i];
}

return 0; // Successo
}

```

Dettagli di ottimizzazione per STM32G4

1. Precisione Singola vs Doppia (float vs double):

- L'STM32G431 integra una **FPU (Floating Point Unit) hardware a singola precisione (float)**. Usare float rende l'esecuzione estremamente veloce (poche decine di cicli di clock per operazione).
- **Attenzione agli overflow di accumulo:** Poiché le somme di grado 4 ($\sum X^4$) crescono rapidamente, assicurati che l'**accumulo** dei campioni a 300 kHz avvenga con variabili `double` o `int64_t`. Converti in `float` solo la matrice finale nel momento in cui la passi alla funzione `Solve_System_5x5`.

2. Memoria e Performance:

- La funzione lavora interamente nello stack usando una piccola matrice aumentata 5×6 , occupando meno di **130 byte di RAM**.
- Il tempo di esecuzione su Cortex-M4 a 170 MHz è di circa $10 \div 15 \mu s$, quindi trascurabile da eseguire a fine misura fuori dalle ISR.

User prompt: più che altro dovresti spiegarmi il metodo, poi per il codice vediamo

Response: Il metodo si basa sull'**Eliminazione di Gauss**, che è la procedura algebrica standard per trasformare un sistema di equazioni complesso in uno semplicissimo da risolvere a cascata (per sostituzione all'indietro).

Nel nostro caso vogliamo risolvere il sistema:

$$\mathbf{M} \cdot \mathbf{x} = \mathbf{V}$$

dove $\mathbf{M}$ è la matrice 5×5 delle somme accumulate, $\mathbf{V}$ è il vettore noto 5×1 e $\mathbf{x} = [A, B, C, D, E]^T$ è il vettore delle nostre 5 incognite.

Il metodo si sviluppa in **3 passaggi concettuali**:

1. Costruzione della Matrice Aumentata

Per non dover applicare le operazioni separatamente sulla matrice $\mathbf{M}$ e sul vettore $\mathbf{V}$, li affianchiamo in un'unica matrice 5×6 , detta **matrice aumentata** $[\mathbf{M} \mid \mathbf{V}]$:

$$\left[\begin{array}{ccccc|c} m_{00} & m_{01} & m_{02} & m_{03} & m_{04} & v_0 \\ m_{10} & m_{11} & m_{12} & m_{13} & m_{14} & v_1 \\ m_{20} & m_{21} & m_{22} & m_{23} & m_{24} & v_2 \\ m_{30} & m_{31} & m_{32} & m_{33} & m_{34} & v_3 \\ m_{40} & m_{41} & m_{42} & m_{43} & m_{44} & v_4 \end{array} \right]$$

Qualsiasi operazione riga applicata a questa matrice preserva esattamente le soluzioni del sistema.

2. Triangolarizzazione Superiore (Forward Elimination)

L'obiettivo di questa fase è annullare tutti i coefficienti sotto la diagonale principale, trasformando la matrice in una **struttura triangolare superiore**:

$$\left[\begin{array}{ccccc|c} m'_{00} & m'_{01} & m'_{02} & m'_{03} & m'_{04} & v'_0 \\ 0 & m'_{11} & m'_{12} & m'_{13} & m'_{14} & v'_1 \\ 0 & 0 & m'_{22} & m'_{23} & m'_{24} & v'_2 \\ 0 & 0 & 0 & m'_{33} & m'_{34} & v'_3 \\ 0 & 0 & 0 & 0 & m'_{44} & v'_4 \end{array} \right]$$

Per fare questo, si procede colonna per colonna (dalla colonna 0 alla colonna 3):

- Selezione del Pivot:** Il punto sulla diagonale corrente m_{kk} si chiama *pivot*.
- Annullamento dei termini sottostanti:** Per ogni riga i sotto il pivot, si calcola un fattore di scala:

$$\text{Fattore} = \frac{m_{ik}}{m_{kk}}$$

- Sottrazione di riga:** Si sottrae dalla riga i la riga del pivot moltiplicata per il Fattore. Questo garantisce che il valore m_{ik} diventi esattamente 0.

Perché si usa il "Pivot Parziale"?

Se durante il processo un elemento sulla diagonale principale m_{kk} è pari a zero (o molto vicino a zero), la divisione $\frac{m_{ik}}{m_{kk}}$ fallisce o genera un enorme errore di arrotondamento numerico. Il **pivot parziale** evita questo problema: prima di elaborare la colonna k , cerca la riga sottostante che ha il valore assoluto più grande in quella colonna e **scambia le due righe**.

3. Sostituzione all'Indietro (Backward Substitution)

Una volta ottenuta la forma triangolare, l'ultima riga rappresenta un'equazione semplicissima con **una sola incognita (E)**:

$$m'_{44} \cdot E = v'_4 \implies E = \frac{v'_4}{m'_{44}}$$

Trovato il valore di E , risaliamo alla quarta riga, che ora contiene solo l'incognita D :

$$m'_{33} \cdot D + m'_{34} \cdot E = v'_3 \implies D = \frac{v'_3 - m'_{34} \cdot E}{m'_{33}}$$

Si continua a risalire riga per riga fino alla prima, ricavando in sequenza ordinata tutti e 5 i parametri:

$$E \longrightarrow D \longrightarrow C \longrightarrow B \longrightarrow A$$

User prompt: Spiega i passaggi matematici per convertire i coefficienti A, B, C, D, E dell'equazione dell'ellisse in Offset X , Offset Y , Guadagni e Sfasamento del sensore.

Response: Partiamo dall'equazione conica implicita che abbiamo risolto:

$$AX^2 + BXY + CY^2 + DX + EY - 1 = 0$$

I segnali reali prodotti da un sensore di posizione (come un encoder magnetico o resolver) con non-idealità sono modellati come:

$$\begin{cases} X = A_x \cos(\theta) + X_0 \\ Y = A_y \sin(\theta + \phi) + Y_0 \end{cases}$$

Dove le nostre incognite fisiche sono:

- X_0, Y_0 : **Offset** dei canali (spostamento del centro dell'ellisse rispetto all'origine $(0, 0)$).
- A_x, A_y : **Guadagni** (ampiezze dei segnali *Sin* e *Cos*).
- ϕ : **Errore di Fase** (scostamento dalla perfetta quadratura di 90°).

Step 1: Calcolo degli Offset (X_0, Y_0)

Il centro dell'ellisse (X_0, Y_0) corrisponde al punto in cui le derivate parziali dell'equazione conica $F(X, Y) = 0$ si annullano contemporaneamente:

$$\begin{cases} \frac{\partial F}{\partial X} = 2AX + BY + D = 0 \\ \frac{\partial F}{\partial Y} = BX + 2CY + E = 0 \end{cases}$$

Risolvendo questo sistema lineare 2×2 per X e Y , otteniamo direttamente le coordinate del centro:

$$X_0 = \frac{2CD - BE}{B^2 - 4AC}$$

$$Y_0 = \frac{2AE - BD}{B^2 - 4AC}$$

Step 2: Traslazione nell'Origine

Definiamo le variabili centrate $u = X - X_0$ e $v = Y - Y_0$. Sostituendo nell'equazione originale, i termini di grado 1 (D ed E) scompaiono:

$$Au^2 + Buv + Cv^2 = G$$

dove il termine noto scalato G vale:

$$G = 1 + AX_0^2 + BX_0Y_0 + CY_0^2 - DX_0 - EY_0$$

Dividendo tutto per G , otteniamo l'ellisse centrata nell'origine:

$$A'u^2 + B'uv + C'v^2 = 1 \quad \text{con } A' = \frac{A}{G}, \quad B' = \frac{B}{G}, \quad C' = \frac{C}{G}$$

Step 3: Calcolo dello Sfasamento (ϕ)

Il termine misto $B'uv$ (o BXY) è presente unicamente a causa del mancato sfasamento a 90° tra *Sin* e *Cos*.

Relazionando i coefficienti conici con il modello del sensore, l'angolo di sfasamento ϕ (espresso in radianti) si ricava direttamente da:

$$\sin(\phi) = -\frac{B'}{2 A' C'}$$

$$\phi = \arcsin\left(-\frac{B}{2 AC}\right)$$

(Se $\phi = 0$, $B = 0$ e i segnali sono perfettamente in quadratura).

Step 4: Calcolo dei Guadagni (A_x, A_y)

Dall'uguaglianza tra il modello fisico del sensore e la forma conica, le ampiezze dei due segnali risultano:

$$A_x = \frac{1}{A' \cdot \cos^2(\phi)} = \frac{G}{A \cdot (1 - \sin^2(\phi))}$$

$$A_y = \frac{1}{C' \cdot \cos^2(\phi)} = \frac{G}{C \cdot (1 - \sin^2(\phi))}$$

Se ti interessa solo il **Guadagno Relativo** (Gain Mismatch $K = \frac{A_x}{A_y}$), la formula si semplifica notevolmente:

$$K = \frac{A_x}{A_y} = \frac{C}{A}$$

Riassunto Formule di Calibrazione

Ottenuti i parametri fisici, la correzione in tempo reale dei campioni grezzi X_{raw}, Y_{raw} si esegue ad ogni passo di controllo con queste semplici operazioni aritmetiche:

1. Rimuovi Offset:

$$X_1 = X_{raw} - X_0$$

$$Y_1 = Y_{raw} - Y_0$$

2. Correggi la Fase:

$$X_2 = X_1$$

$$Y_2 = \frac{Y_1 - X_1 \cdot \sin(\phi)}{\cos(\phi)}$$

3. Normalizza i Guadagni:

$$X_{corr} = \frac{X_2}{A_x}$$

$$Y_{corr} = \frac{Y_2}{A_y}$$

I punti (X_{corr}, Y_{corr}) formeranno ora un **cerchio perfetto di raggio 1** centrato nell'origine, pronto per il calcolo dell'angolo preciso via CORDIC_Atan2.

User prompt: Fornisci un esempio di codice C per applicare la correzione di offset, gain e fase in tempo reale ai campioni ADC prima di passarli al CORDIC dell'STM32G431.

Response: Ecco un esempio completo in C ottimizzato per l'STM32G431.

La routine di correzione sfrutta le funzioni intrinseche del **Cortex-M4F** e l'hardware **CORDIC** di STM32 per elaborare ogni coppia di campioni in pochissimi cicli di clock.

1. Struttura dei parametri di calibrazione

Per ridurre al minimo i calcoli a 300 kHz, **pre-calcoliamo** i coefficienti di correzione una sola volta a fine calibrazione (fuori dall'interrupt):

```
#include "stm32g4xx_hal.h"
#include <math.h>

// Struttura dei parametri pre-calcolati (stoppati in RAM o Flash)
typedef struct {
    float offset_x;      // X0 (Offset Sin)
    float offset_y;      // Y0 (Offset Cos)
    float inv_gain_x;    // 1.0f / Ax
    float inv_gain_y;    // 1.0f / Ay
    float tan_phi;       // tan(phi) per la correzione di fase
    float inv_cos_phi;   // 1.0f / cos(phi)
```

```

} Calib_Params_t;

static Calib_Params_t calib;

/**
 * @brief Pre-calcola le costanti di correzione a partire dai parametri dell'ellisse.
 *        Da chiamare UNA SOLA VOLTA dopo aver risolto il sistema 5x5.
 */
void Calib_Update_Constants(float X0, float Y0, float Ax, float Ay, float phi)
{
    calib.offset_x    = X0;
    calib.offset_y    = Y0;
    calib.inv_gain_x  = 1.0f / Ax;
    calib.inv_gain_y  = 1.0f / Ay;

    // Semplificazione algebrica della matrice di correzione di fase
    calib.tan_phi     = tanf(phi);
    calib.inv_cos_phi = 1.0f / cosf(phi);
}

```

2. Routine di Correzione in Tempo Reale (Inline Assembly / FPU)

La correzione applica le seguenti trasformazioni algebriche sui dati grezzi (X_{raw} , Y_{raw}):

- Rimozione Offset & Guadagno X:** $X_c = (X_{raw} - X_0) \cdot \frac{1}{A_x}$
- Rimozione Offset Y:** $Y_{temp} = (Y_{raw} - Y_0) \cdot \frac{1}{A_y}$
- Disaccoppiamento di Fase & Guadagno Y:** $Y_c = \frac{Y_{temp} - X_c \cdot \sin(\phi)}{\cos(\phi)}$

```

/**
 * @brief Applica la correzione di Offset, Gain Mismatch ed Errore di Fase.
 *
 * @param raw_x Campione grezzo SIN dall'ADC
 * @param raw_y Campione grezzo COS dall'ADC
 * @param corr_x Puntatore al valore SIN corretto (Output [-1.0, 1.0])
 * @param corr_y Puntatore al valore COS corretto (Output [-1.0, 1.0])
 */
__STATIC_INLINE void Apply_Correction(int16_t raw_x, int16_t raw_y, float *corr_x, float *corr_y)
{
    // 1. Rimozione Offset
    float x1 = (float)raw_x - calib.offset_x;
    float y1 = (float)raw_y - calib.offset_y;

    // 2. Normalizzazione Guadagno
    float x2 = x1 * calib.inv_gain_x;
    float y2 = y1 * calib.inv_gain_y;

    // 3. Correzione Sfasamento (Semicompensazione di Gram-Schmidt)
    *corr_x = x2;
    *corr_y = (y2 - (x2 * calib.tan_phi)) * calib.inv_cos_phi;
}

```

3. Invio al CORDIC dell'STM32G431

L'acceleratore CORDIC accetta numeri in formato **Fixed-Point Q1.31** (valori compresi tra -1.0 e $+1.0$). Usiamo l'istruzione hardware `__SCALE_SAT` o una semplice conversione per preparare i dati.

```

extern CORDIC_HandleTypeDef hcordic;

/**
 * @brief Passa le coordinate corrette al CORDIC in modalità Zero-Overhead.
 * @return float Angolo calcolato in radianti [-PI, +PI]
 */
__STATIC_INLINE float Process_CORDIC_Atan2(float x_corr, float y_corr)
{
    // Converti da float [-1.0, 1.0] a Fixed-Point Q1.31 (int32_t)
    int32_t q31_x = (int32_t)(x_corr * 2147483648.0f);
    int32_t q31_y = (int32_t)(y_corr * 2147483648.0f);
}

```

```

int32_t q31_angle;

// Scrittura diretta sui registri CORDIC (evita il footprint dell'HAL in ISR)
// CORDIC WDATA: carica Y e X consecutivamente
hcordic.Instance->WDATA = (uint32_t)q31_y;
hcordic.Instance->WDATA = (uint32_t)q31_x;

// Lettura del risultato (Angolo in formato Q1.31 normalizzato a [-PI, +PI])
q31_angle = (int32_t)hcordic.Instance->RDATA;

// Converti l'angolo Q1.31 in radianti float (-PI a +PI)
return ((float)q31_angle / 2147483648.0f) * 3.14159265358979323846f;
}

```

4. Integrazione nella routine d'interruzione DMA (ISR @ 300 kHz)

Ecco come viene concatenato il flusso all'interno dell'interrupt del buffer DMA dell'ADC:

```

// Buffer DMA gestito in Dual-Mode (ADC1 = Sin, ADC2 = Cos)
extern uint32_t adc_dma_buffer[64];

void DMA1_Channel1_IRQHandler(void)
{
    // Esempio: Elaborazione sulla prima metà del buffer (Half Transfer)
    if (__HAL_DMA_GET_FLAG(NULL, DMA_FLAG_HT1))
    {
        __HAL_DMA_CLEAR_FLAG(NULL, DMA_FLAG_HT1);

        for (uint8_t i = 0; i < 32; i++)
        {
            // Estratti i due canali a 12 bit
            int16_t sin_raw = (int16_t)(adc_dma_buffer[i] & 0xFFFF);
            int16_t cos_raw = (int16_t)((adc_dma_buffer[i] >> 16) & 0xFFFF);

            float sin_corr, cos_corr;

            // 1. Correzione Ellisse (Offset, Gain, Fase)
            Apply_Correction(sin_raw, cos_raw, &sin_corr, &cos_corr);

            // 2. Calcolo Angolo HW via CORDIC
            float angle_rad = Process_CORDIC_Atan2(sin_corr, cos_corr);

            // 3. Usa angle_rad per il controllo motore / tracciamento posizione
            // ...
        }
    }
}

```

Vantaggi di questo approccio per STM32G4

- 1. Zero Allocazione RAM dinamica:** Vengono usati unicamente i registri della FPU e dello stack locale.
 - 2. Cicli di Clock Minimi:** L'intera funzione `Apply_Correction + Process_CORDIC_Atan2` richiede meno di $30 \div 40$ cicli di clock (a 170 MHz significa $\approx 0.2 \mu s$, lasciando oltre il 90% di tempo CPU libero per altri compiti).
 - 3. Accesso Diretto ai Registri CORDIC:** Bypassando `HAL_CORDIC_Calculate()` si eliminano tutti i controlli software `HAL_LOCK` inutili nelle routine a 300 kHz.
-